Hands-On PaaS ToDoApp

To-Do´s

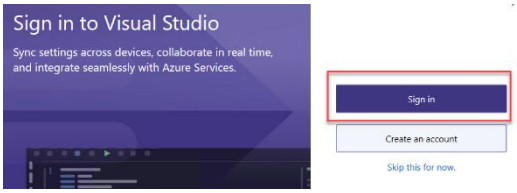
Prerequisites

- ☐ **1.** Ensure that you have access to an active Azure Student Subscription. You can get it by calling the following URL and follow the instructions: https://azure.microsoft.com/en-us/free/students/
  IMPORTANT: Use your FHNW account (e.g. Thomas.etteler@students.fhnw.ch)!
- ☐ **2.** Ensure that the source code of the Demo application is stored at C:\AzureDemoContent on your VM. If not, download it from this location: https://github.com/OliCSADoerr/ServerlessToDo
- ☐ **3.** Ensure that the Storage Explorer is installed on your VM and started at least once (the first start will take a little bit).
- ☐ **4.** Ensure that Visual Studio is installed on your VM.
- ☐ **5.** Ensure that your Browser is displaying the Azure Portal in English
- **6.** MAC Users ONLY: Ensure that you have downloaded the "Windows App" from the Mac App Store: Windows App on the Mac App Store

General Settings

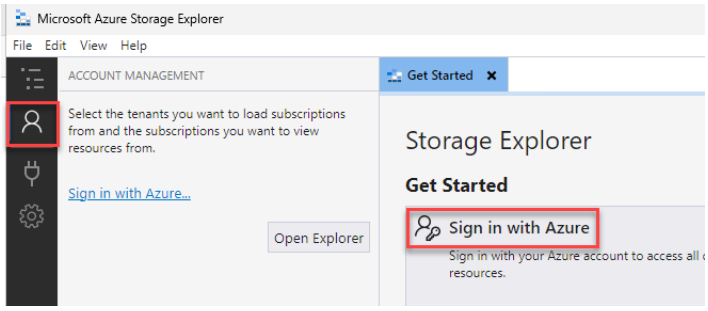
- ☐ **1.** Due to FHNW Subscription restriction reasons, please create all Azure resources within the **Region Poland Central**.
- ☐ **2.** All resources may require a three-digit identifier, e.g. JSM for **J**ohn **Sm**ith. Please think of your identifier before you start with creating resources, it will save you time.

☐ **3.** Ensure that you have provided your FHNW account which is the Azure Subscription owner within Visual Studio 2022 (NOT Visual Studio Code!) on your VM. Click Sign-in and follow the instructions on the screen.



☐ **4.** Ensure that you have provided your FHNW account, which is the Azure Subscription owner, within Storage Explorer on your VM.

Select "Subscription" and "Azure" afterwards and click Next. Follow the instructions on the screen.



Serverless App – Action items

1. Connect to your Hands-On-Lab VM - ~1 Min.

Start the RDP Client (e.g. "mstsc" on Windows / MacUser: Windows App) on your Notebook and connect to the IP-Address/Port that has been provided in the PDF next to your Number you have grabbed on paper. Ensure you are not connected to any corporate VPN which might block RDP connections.

Please use the following Login Credentials:

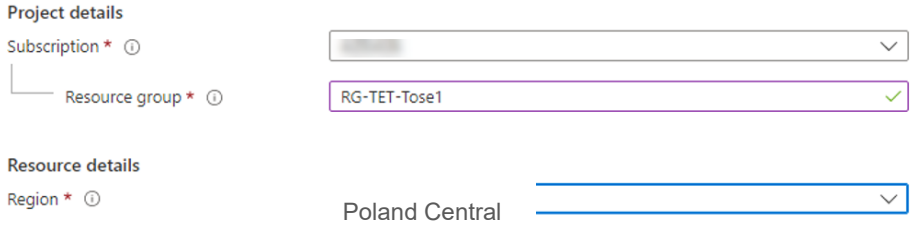
Username: Cloud-Handson

Password: FHnw#1n-Cl4d

2. Create your resource group - ~3 Min.

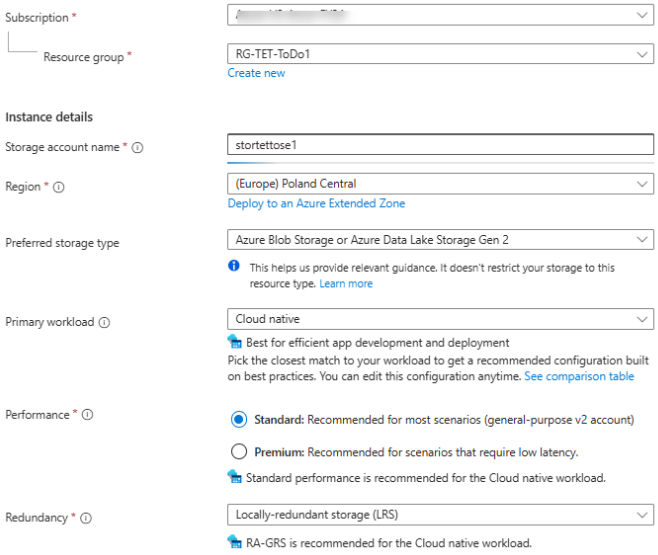
Open Chrome or Edge and call the Azure Portal at https://portal.azure.com. Login with your FHNW account When you are logged in, create a resource group with the name «RG-<your-3-digit>-Tose1» using the Azure portal search bar. The image below can be used as an example.

ALWAYS click the "Create" Button on the "Review+Create" screen!



3. Provide Storage - ~5 Min.

Within the newly created resource group, create a new storage account. In the upper left corner of the Resource Group click “Create” and search for “storage account”. Take the first result.

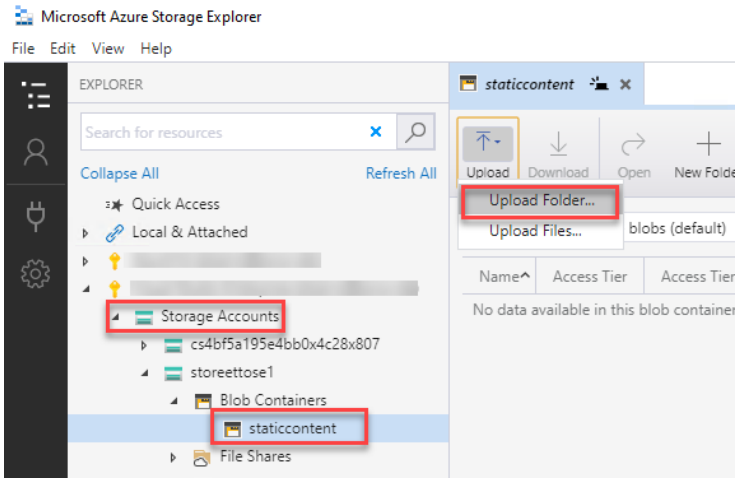


Use the default settings, except the following changes
Name: stor**<JSM>**tose1
Pref. storage type: Blob
Prim. Workl.: Cloud native
When you click through the next options – which can stay in default – **notice** what a huge variety of options and configurations an easy resource like a storage account already provides in the Cloud.

After the storage account has been created, go to its overview page (e.g. Click on “Go to resource”) and enable the “Blob anonymous access” setting in the section “Blob service”. When you click on “Disabled” you will be redirected to the feature overview page. Here, select “Enabled” for the setting “Allow Blob anonymous access” and click save on the top left of the site.
Navigate to Data storage -> Containers and setup a new container with those values:

Name:	staticcontent
Access:	Container (anonymous) (if you can´t select it – you haven´t activated the Blob anonymous access beforehand)

On your VM use the “ Azure Storage Explorer” to upload the folder “ui” located at C:\AzureDemoContent (Path in your VM) as seen in the screenshot below. The Azure Storage explorer can be found on the Desktop / Taskbar of your VM.
The folder and its content should be stored in the newly created container “staticcontent”.



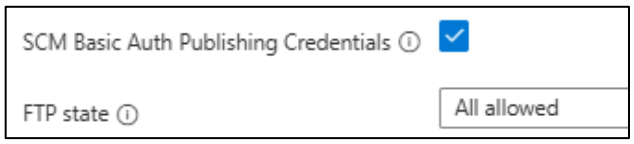
Ensure you have saved the URL to this newly created folder, since you need it for a later step.
You could find it e.g. in the properties (right click of the file) of the index.html file within the “ui” folder.

4. Setup Azure Function - ~5 Min.

Use the search bar in the middle of the top of the Azure Portal and search for a “Function App”. Start creating it, by clicking “Create” on the top left corner. Use the default settings in the recommended Azure Region from the beginning. Always use the “Next” Button, not directly the “Review+Create”.

Hosting Plan: Consumption (the option far right!)
Resource group: Step 2
Func.App-Name: fap-**<JSM>**-tose1
Secure Hostname: Disable
Runtime stack: .NET
Version: 8 LTS (isolated)
Storage account (page 2): From Step 3

Once your Function App is created, change the following settings under Settings -> Configuration -> General Settings. Those two settings have to match the status which you can see in the screenshot. Once done, click “Apply” on the bottom of the site!



5. Create a Cosmos DB Account ~ 5 Min.

Staying in the Azure Portal, use the search bar on top to create a “Cosmos DB”. Use the “Create” button to create a “Cosmos DB for NoSQL” (first option) with the default settings.

Workload Type: Learning
Resource-Group: Step 2
Account-Name: cos-<JSM>-tose1
Region: The one that is available for you to select (if you don´t see any – click the “Enable” / “Disable” Radio Button in the Availability Zone area)

>>> THINK ABOUT TAKING A BREAK – NOW?! The Cosmos DB takes ~ 7 Min. to create. <<<

After the CosmosDB Account has been created, ensure that the option "Public network access" is set to "All networks" in Settings -> Networking.

6. Create a Cosmos DB Container - ~5Min

Create a new Cosmos DB Container for .NET within the Azure Portal with default settings. This can be done by e.g. using the “Data Explorer”, which can be found in the left navigation pane of the Cosmos DB resource.

ATTENTION: All values are CaSe-SeNSItive!

Database id: ServerlessTodo
Container id: TodoItems
Partition Key: /ItemOwner
Advanced Settings Activate “[…]older Cosmos.NET SDK versions[…]”

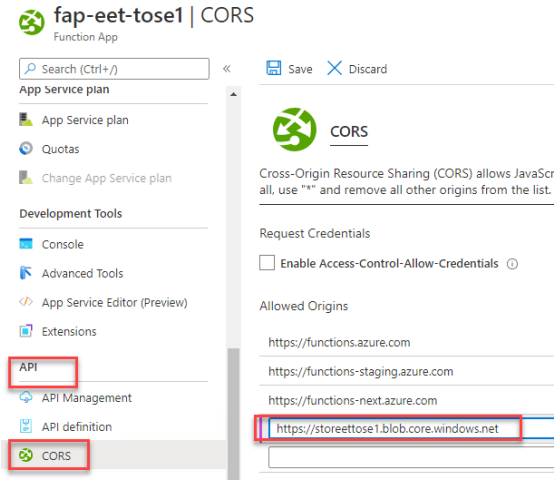
7. Connect the Cosmos DB and the Function App - ~5 Min.

To ensure that you App can reach the Database, you need to ensure that the connection string is stored as an environment variable of the Function Runtime.
Therefore copy the "primary **connection** string" of the CosmosDB account (you will get this in the section Settings -> *Keys* within the left navigation pane of the CosmosDB resource). Add this value to the Function App under Settings -> Environment variables:

Name: CosmosDBConnectionString
Value: <CosmosDB connection string>

Click "Apply", after you have made the changes.
BE aware of the fact that the values are all CaSe seNsITive!

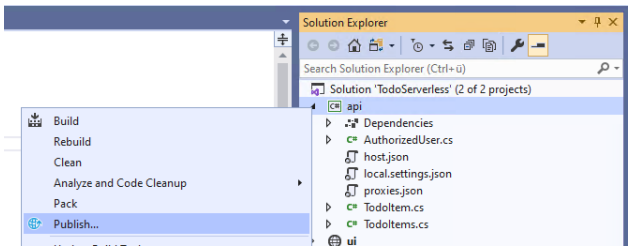
Configure CORS – Function App - ~3 Min.
Configure the CORS setting of the FunctionApp, to allow connections from the website which is hosted on the storage account to the FunctionApp. The required value could be found in Step 3 (URL of index.html, but excluding the URI!).
Click "Save" within the FunctionApp Settings, after you have made the changes.



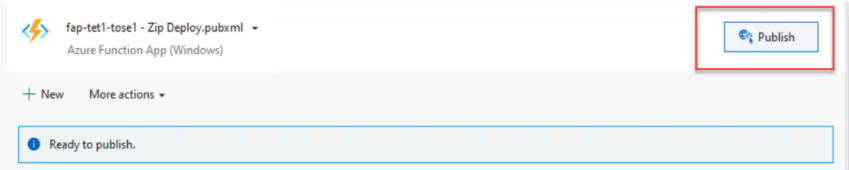
8. Deploy your FunctionApp to Azure - ~20 Min.

Start Visual Studio on your VM with the option "Open a Project" and open the file C:\AzureDemoContent \TodoServerless.sln. When the dialog appears that asks you if you trust the files since they are downloaded from the web, click "Trust and Continue".

Publish the solution «api» to your recently created Azure Function (fap-<tet>-tose1).
To do so, right click on "api" within the solution explorer and select "Publish". Follow the instructions on the screen for publishing to an "Azure Function App (Windows)" (You might have to click on "Add a publish profile").

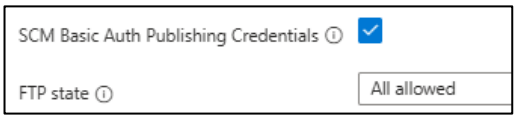


Finally you have to "publish" your Function to Azure by clicking the button "Publish".



In the dialog – that might pop-up! - and asks you to provide a username and password, please select "Attempt to retrieve credentials from Azure" on the bottom left corner. Once you got the credentials click "Save". The publishing process should continue and finish successfully.

If you get an error that is related to an “Unauthorized Access” – ensure that those two settings in the Function App are still configured as seen in the screenshot:

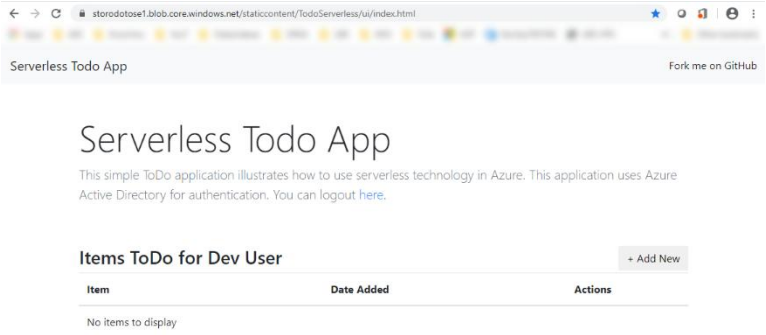


9. Update the Javascript File - ~5 Min

In the Azure Portal, navigate to the overview page of your Function App and copy the value of “Default domain” which can be found in the top right corner.
Navigate to the storage account->Data storage->Containers and select the file called "staticcontent/ui/js/*vars.js*", since it needs some properties to be updated. In the newly appearing window on the side you see in the tabs – on the top - the option “Edit” which you should select.
Replace “<YOUR_FUNCTIONS_URL>” with he value you have just copied from the Default Domain in the Function app (e.g. var remoteUrl = "https://fap-tet-tose1.azurewebsites.net/api/todoitem";).
Click “Save” in the upper left corner.

10. See your App running - ~ 3 Min.

Open your Browser and call the URL of the index.html file which is located in your storage account. You have stored the URL as a last step in Task 3.
Add some ToDo items, after that delete one or two of them. If that´s successful, you have tested that your FunctionApp and the CosmosDB connection are working properly.



!!! CONGRATS !!!

If you have some time left

- check out the metrics in the Application Insights App
- think of possible Authentication issues and
- how this operation could be automated.

If you are done with it – you can delete the whole resource group of “Tose1”